Group 28 Progress Report: Spoiler Review Classifier

Sky Quan, Aidan Lao, Hanna Arendt
{quans6,laoa6,arendth}@mcmaster.ca

1 Introduction

Many people browse reviews before starting a book. There are popular platforms like Goodreads to facilitate this, but reviews often contain major spoilers. This can ruin the experience of reading the book and upset readers that just want to hear a high-level overview or opinions. Goodreads lets reviewers hide this content behind spoiler tags, but it is based on trust, since humans decide whether to include the spoiler tags or not. Because of this, readers can have a book spoiled before they ever open it even if it doesn't have a spoiler tag. So, creating a spoiler detector keeps readers safe by checking for untagged spoilers before they are posted, letting readers safely filter through reviews that don't have spoilers. This would make review platforms safer for readers to get a high-level overview of a book without worrying about being spoiled.

2 Related Work

Here, talk about the related work you encountered for your approach. Cite at least 5 references. Refer to item 2. No one has done exactly your task? Write about the most similar thing you can find. This should be around 0.25-0.5 pages.

3 Dataset

The source dataset is the Goodreads Spoilers Dataset, originally collected and released by McAuley Lab at UC San Diego, and reposted on Kaggle by Ahmad Mujtaba.

- Original source: https://cseweb.ucsd.edu/~jmcauley/datasets/goodreads.html
- Kaggle link: https://www.kaggle.com/datasets/pypiahmad/goodreads-book-reviews

The data was collected in late 2017 from goodreads.com by scraping users' public Goodreads shelves. Given the dataset size of 1,378,033 reviews (of 25,475 books by 18,892 users), we use a smaller, randomly sampled subset of data composed of 10,000.

To obtain this smaller and randomized subset, we used stratified reservoir sampling. This was used for two main reasons. First, to give every datapoint a randomized chance to appear in our smaller subset. Secondly, to combat the class imbalance within the source dataset. Since the dataset contains many more reviews without spoilers compared to those with, randomly sampling datapoints doesn't give the model enough spoilers to train on. So, stratified reservoir sampling allows us to also sample a predefined number of spoilers and non-spoilers. With the parameters we set, we create a dataset with 10,000 reviews where 20% of the reviews contain spoilers and 80% of the reviews do not, regardless of class imbalance.

The source dataset does not label each review as containing or spoiler or not. However, every spoiler within the dataset contains the text `(view spoiler)[...(hide spoiler)]`, where `...` represents the specific spoiler to each book. So, if a review contains the text `(view spoiler)` or `(hide spoiler)`, it was labelled as 1, contains a spoiler, and if it did not contain the text, it was labelled as 0, contains no spoiler. The labels were placed under the newly created column has_spoiler.

The dataset is then cleaned by removing the unnecessary columns ['userid', 'rating', 'date_added', 'date_updated', 'read_at', 'started_at', 'n_votes', 'n_comments'] while keeping the necessary columns ['review_id', 'book_id', 'review_text', 'has_spoiler']. The spoiler tags `(view spoiler)[` and `(hide spoiler)]` are then removed from review_text. Finally, the dataset is split into training, validation, and testing data by book, with 80%, 10%, and 10% of the

books respectively, so that all reviews of a given book appear in only one of the three sets (see Section 5).

Two example datapoints, with review_text shortened for readability:

```
{'book_id': '16981',
 'review_id':
   'a5d2c3628987712d0e05c4f90798eb67',
 'review_text': 'Recommended by Don Katz.
   Avail for free in December...',
 'has_spoiler': 0}

{'book_id': '28684704',
 'review_id':
   '2ede853b14dc4583f96cf5d120af636f',
 'review_text': '... can't be discussed
   without spoilers ... Daniela says, ...',
 'has_spoiler': 1}
```

4 Features

Each review is represented as a bag-of-words vector weighted by TF-IDF (term frequency–inverse document frequency) (Salton and Buckley, 1988), built with scikit-learn's `TfidfVectorizer` (Pedregosa et al., 2011). A term gets a high weight when it appears often in a review but rarely across the training set, so common filler words count for little and more specific words such as *killed* or *ending* count for more. We use the following settings:

- **Unigrams and bigrams** (`ngram_range=(1,2)`). Many spoiler cues are short phrases rather than single words, e.g. *the end* or *she dies*.
- **Lowercasing**, so *Ending* and *ending* are the same feature.
- **No stop-word removal.** Pronouns such as *he* and *she* turned out to be among the strongest spoiler features, since spoilers tend to retell what characters did. Removing them would throw this signal away.
- **`min_df=2`** drops terms that appear in only one review. These are mostly typos and one-off names that cannot generalize.
- **`max_df=0.9`** drops terms that appear in more than 90% of reviews.
- **Sublinear term frequency** $(1 + \log \mathrm{tf})$, so a word repeated many times in a long review does not dominate the vector.
- **Vocabulary capped at 50,000 terms**, keeping the most frequent ones.

The vectorizer is fit on the training set only and then applied to the validation and test sets. This way, words that only occur in held-out books do not enter the vocabulary, which matches how the model would be used on new books.

We do not do any further feature selection, augmentation or learned embeddings yet. Masking book-specific words (character and place names) is planned as the next feature step (Section 7).

Split	Reviews	Books	Spoiler %
Train	8,015	5,108	20.1
Validation	1,014	639	20.3
Test	971	639	18.8

Table 1: Book-based split. No book appears in more than one set.

5 Implementation

All code is written in Python with pandas and scikit-learn and is in `models/train_logreg.py`.

Splitting by book. We split the 10,000 reviews into training, validation and test sets with `GroupShuffleSplit`, using `book_id` as the group. First 80% of the books go to training, then the remaining books are split evenly between validation and test. Every review of a given book therefore appears in exactly one set, and the script checks that no book is shared between sets. This tests whether the model can find spoilers in reviews of books it has never seen, rather than memorizing plot details of specific books. Because books have different numbers of reviews, the split is close to, but not exactly, 80/10/10 by review count. Table 1 shows the result. The spoiler share stays close to 20% in every set.

Majority-class baseline. The baseline always predicts the most common class in the training set, which is "no spoiler". It reaches 79.7% accuracy on the validation set while finding no spoilers at all, so its spoiler-class F1 is 0. This shows why accuracy alone is misleading for this task.

Logistic regression. Our main model is L2-regularized logistic regression on the TF-IDF features. It models the probability that a review contains a spoiler as $p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ and is trained by minimizing the regularized cross-entropy (log) loss

$$\frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \alpha_{y_i}\, \ell\big(y_i, \sigma(\mathbf{w}^\top \mathbf{x}_i + b)\big), \quad (1)$$

where ℓ is the log loss, C controls the strength of regularization (smaller C means stronger reg-

Weights	C	P	R	F1
Equal	0.1	0.00	0.00	0.00
Equal	1	0.60	0.18	0.27
Equal	10	0.51	0.28	0.36
Balanced	0.1	0.37	0.70	0.48
Balanced	1	0.41	0.63	**0.49**
Balanced	10	0.46	0.50	0.48

Table 2: Validation precision (P), recall (R) and F1 on the spoiler class for a subset of the tuning grid.

ularization), and α_y is an optional per-class weight. The model is optimized with scikit-learn's `liblinear` solver, a coordinate-descent method suited to sparse, high-dimensional text features. A review is predicted as a spoiler when $p(y = 1 \mid \mathbf{x}) \geq 0.5$.

Tuning. We tuned two hyperparameters on the validation set: $C \in \{0.01, 0.1, 1, 10, 100\}$ and the class weights, either equal or *balanced*. Balanced weights scale each class inversely to its frequency, so a missed spoiler costs about four times as much as a false alarm. We picked the setting with the highest spoiler-class F1 on validation (Table 2). The test set was not used for any of these choices.

Class weighting mattered far more than C. With equal weights and strong regularization, the model collapses to the baseline and predicts no spoilers. Even at its best, it finds fewer than a third of the spoilers. Balanced weights roughly double recall at the cost of some precision, and every balanced setting beats every equal-weight setting on F1. The selected model uses balanced weights and $C = 1$, reaching a validation F1 of 0.49 (precision 0.41, recall 0.63), against 0 for the baseline.

Inspecting the learned weights shows that the strongest spoiler features are plot-retelling words: *killed*, *died*, *ending*, *the end*, *scene*, *cheating*, and the pronouns *he* and *she*. The strongest non-spoiler features are about the review itself rather than the plot: *read*, *review*, *blog*, *publisher*, *http*. Interestingly, *spoiler alert* points toward non-spoiler, likely because reviewers use the phrase to promise a spoiler-free review.

6 Results and Evaluation

How are you evaluating your model? What results do you have so far? What are your baselines? Refer to item 5. This may take around 0.5 pages.

7 Feedback and Plans

Write about your plans for the remainder of the project. This should include a discussion of the feedback you received from your TA, and how you plan to improve your approach. Reflect on your implementation and areas for improvement. Refer to item 6. This may be around 0.5 pages.

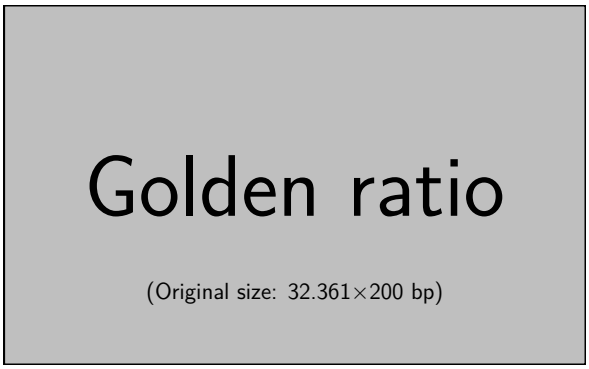


Figure 1: A figure with a caption that runs for more than one line. Example image is usually available through the `mwe` package without even mentioning it in the preamble.

8 Template Notes

You can remove this section or comment it out, as it only contains instructions for how to use this template. You may use subsections in your document as you find appropriate.

8.1 Tables and figures

See Table 3 for an example of a table and its caption. See Figure 1 for an example of a figure and its caption.

8.2 Citations

Table 3 shows the syntax supported by the style files. We encourage you to use the natbib styles. You can use the command `\citet` (cite in text) to get "author (year)" citations, like this citation to a paper by Gusfield (1997). You can use the command `\citep` (cite in parentheses) to get "(author, year)" citations (Gusfield, 1997). You can use the command `\citealp` (alternative cite without parentheses) to get "author, year" citations, which is useful for using citations within parentheses (e.g. Gusfield, 1997).

8.3 References

Many websites where you can find academic papers also allow you to export a bib file for citation or bib formatted entry. Copy this into the `custom.bib` and you will be able to cite the paper in the LaTeX. You can remove the example entries.

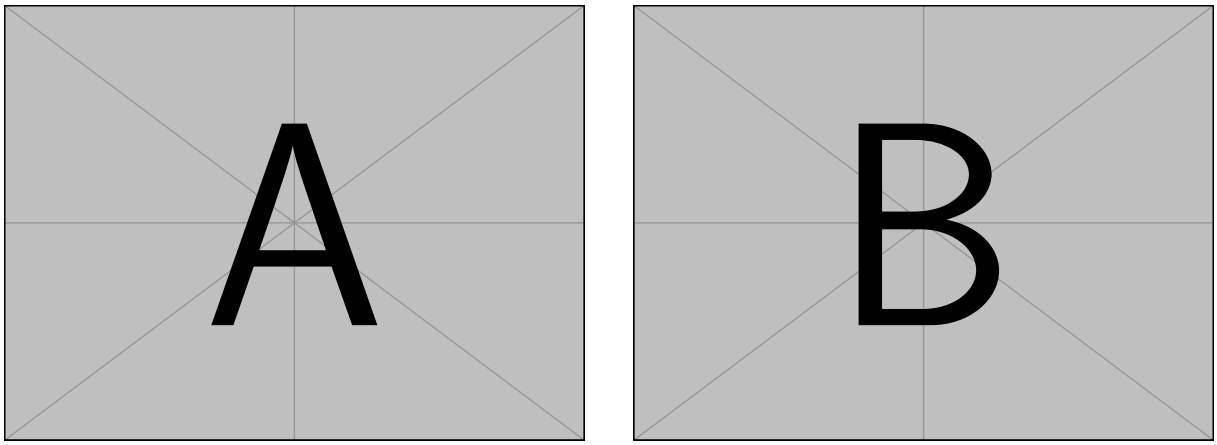


Figure 2: A minimal working example to demonstrate how to place two images side-by-side.

Output	natbib command	ACL only command
(Gusfield, 1997)	`\citep`	
Gusfield, 1997	`\citealp`	
Gusfield (1997)	`\citet`	
(1997)	`\citeyearpar`	
Gusfield's (1997)		`\citeposs`

Table 3: Citation commands supported by the style file.

8.4 Equations

An example equation is shown below:

$$A = \pi r^2 \tag{2}$$

Labels for equation numbers, sections, subsections, figures and tables are all defined with the `\label{label}` command and cross references to them are made with the `\ref{label}` command. This an example cross-reference to Equation 2. You can also write equations inline, like this: $A = \pi r^2$.

Team Contributions

Write in this section a few sentences describing the contributions of each team member. What did each member work on? Refer to item 7.

Sky Quan: Handled everything involving the dataset. Loaded the source dataset and completed all the preprocessing: cleaning, labelling, and spliting the dataset as required. Wrote the Introduction and Dataset section.

References

Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6:1817–1853.

Galen Andrew and Jianfeng Gao. 2007. Scalable training of L1-regularized log-linear models. In *Proceedings of the 24th International Conference on Machine Learning*, pages 33–40.

Dan Gusfield. 1997. *Algorithms on Strings, Trees and Sequences*. Cambridge University Press, Cambridge, UK.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Mohammad Sadegh Rasooli and Joel R. Tetreault. 2015. Yara parser: A fast and accurate dependency parser. *Computing Research Repository*, arXiv:1503.06733. Version 2.

Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523.